

Appendix B.2

Designing Classes, Composition, and Inheritance Awareness

30 minutes instruction + 30 minutes exercises

```
object.attribute
```

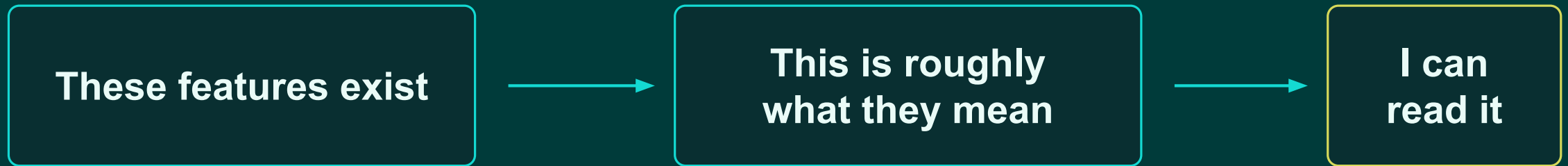
```
object.method()
```

```
Class → Object
```

```
HAS-A / IS-A
```

The Goal Is Recognition

Students should understand the shape of object-oriented code without mastering architecture.



Not the goal: designing sophisticated object-oriented architectures.
The goal: recognizing composition, inheritance, overriding, `super()`, type hints, and tradeoffs.

Review: State + Behavior

Appendix B.1 named the basic pieces.

```
class Contact:

    def __init__(self, name, email):
        self.name = name
        self.email = email

    def display(self):
        print(self.name)

alice = Contact("Alice",
               "alice@example.com")
alice.display()
```

Contact
class

alice
object / instance

self.name
attribute

display()
method

Mental model:
object = data stored on it + methods
that work with that data

Classes Can Hide Details

The caller asks for the behavior, not the step-by-step implementation.

```
class Contact:

    def normalize(self):
        self.name = self.name.strip()
        self.email = (
            self.email
            .strip()
            .lower()
        )

contact.normalize()
```

Caller

What do I
want?

Object method
How is it done?

This is abstraction: useful behavior
behind a clear method name.

Type Hints Can Refer to Classes

A custom class can be used where int, str, or float would appear.

```
def print_contact(  
    contact: Contact  
) -> None:  
  
    print(contact.name)  
  
contacts: list[Contact] = []
```

contact: Contact

This function expects
a Contact object.

list[Contact] means:
this list should contain Contact
objects.

isinstance()

Python can ask whether an object belongs to a type.

```
if isinstance(contact, Contact):  
    print("This is a Contact")  
  
if isinstance(value, int):  
    print("This is an integer")
```

Built-in classes:
int, str, list, dict

Custom classes:
Contact, Address, EmployeeContact

Same idea, different class.

Composition: Objects Containing Objects

Composition models things that naturally contain other things.



Composition is usually described as a “has-a” relationship.
Contact has an Address. Order has LineItems. Car has an Engine.

Create an Address Object

Address becomes its own small type with related data.

```
class Address:

    def __init__(
        self,
        street,
        city,
        state,
        zip_code
    ):
        self.street = street
        self.city = city
        self.state = state
        self.zip_code = zip_code
```

Address object state

street

city

state

zip_code

One object can become data for another object.

Contact Has an Address

The Contact stores an Address object as one of its attributes.

```
class Contact:

    def __init__(
        self,
        name,
        email,
        address
    ):
        self.name = name
        self.email = email
        self.address = address
```

```
address = Address(
    "100 Main Street",
    "Pensacola",
    "FL",
    "32501"
)

contact = Contact(
    "Alice",
    "alice@example.com",
    address
)
```

```
contact.address.city →
    "Pensacola"
```

Nested Objects Feel Like Nested Dictionaries

Composition extends an idea students already know.

nested dictionary

```
contact = {  
    "name": "Alice",  
    "address": {  
        "city": "Pensacola",  
        "state": "FL",  
    },  
}  
  
contact["address"]["city"]
```

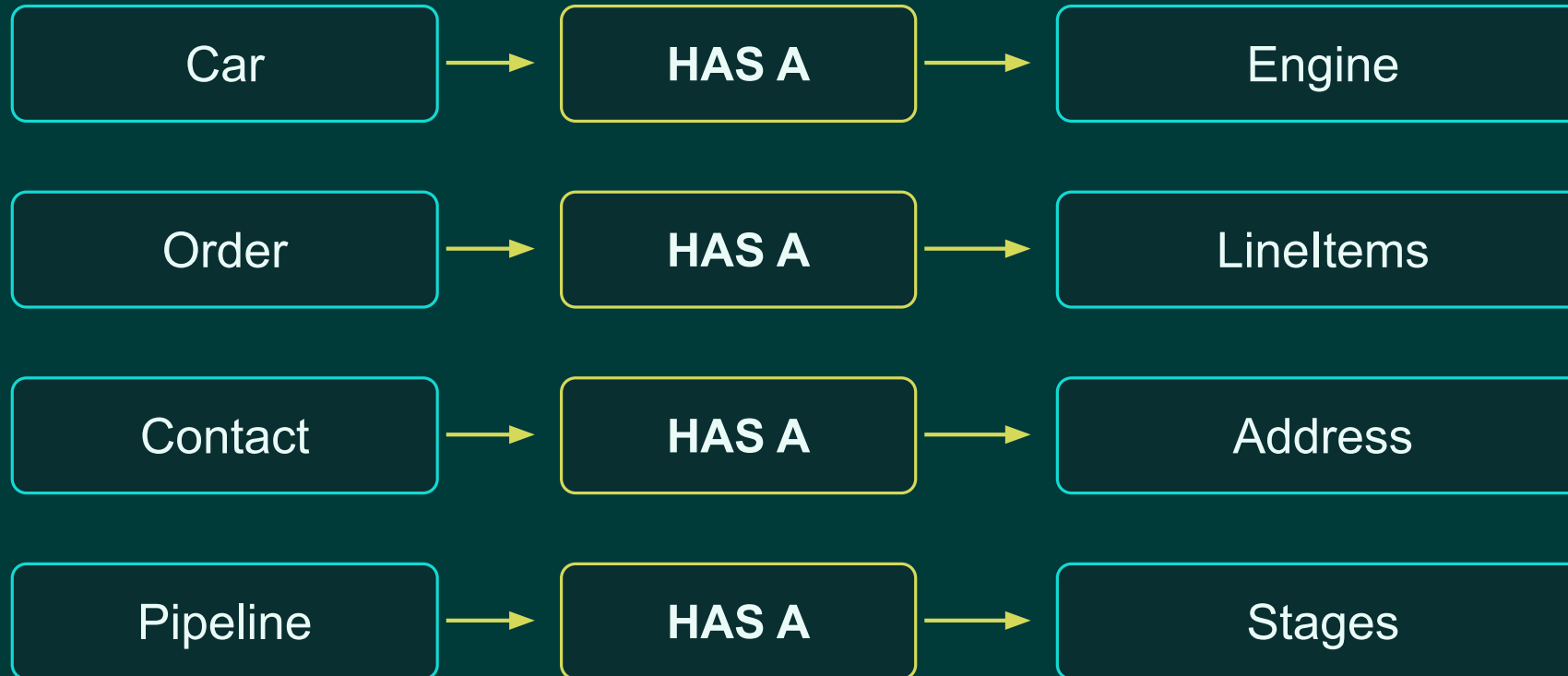
nested objects

```
address = Address(  
    "100 Main Street",  
    "Pensacola",  
    "FL",  
    "32501"  
)  
  
contact.address.city
```

Different syntax, same relationship: one thing contains another thing.

Why Composition Matters

Programs often model things made from smaller pieces.



Use composition when pieces are meaningful concepts by themselves.

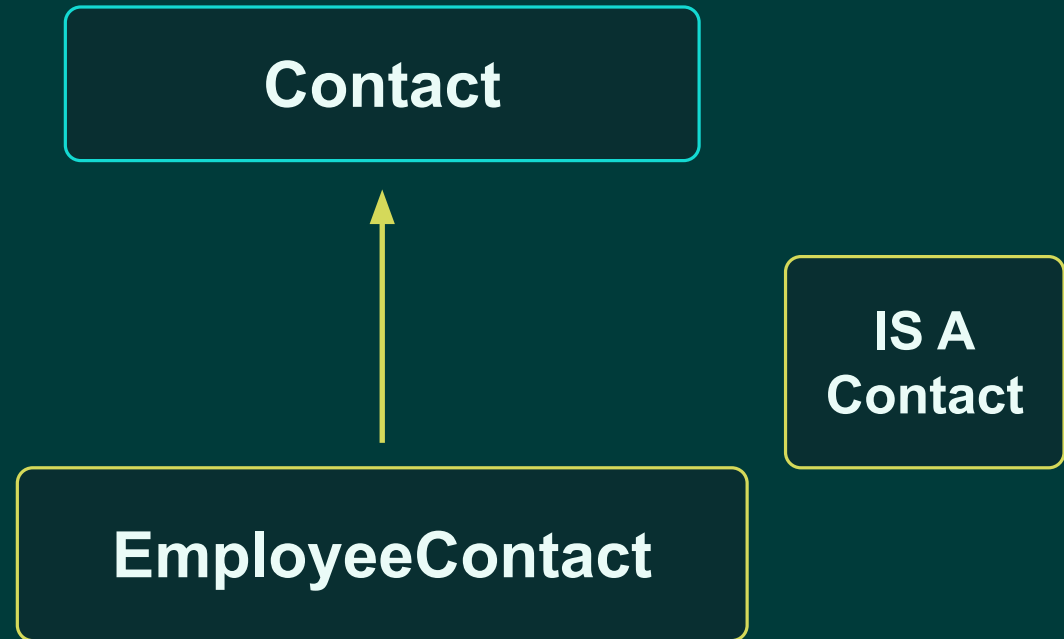
Inheritance

Inheritance lets one class build on another class.

```
class Contact:

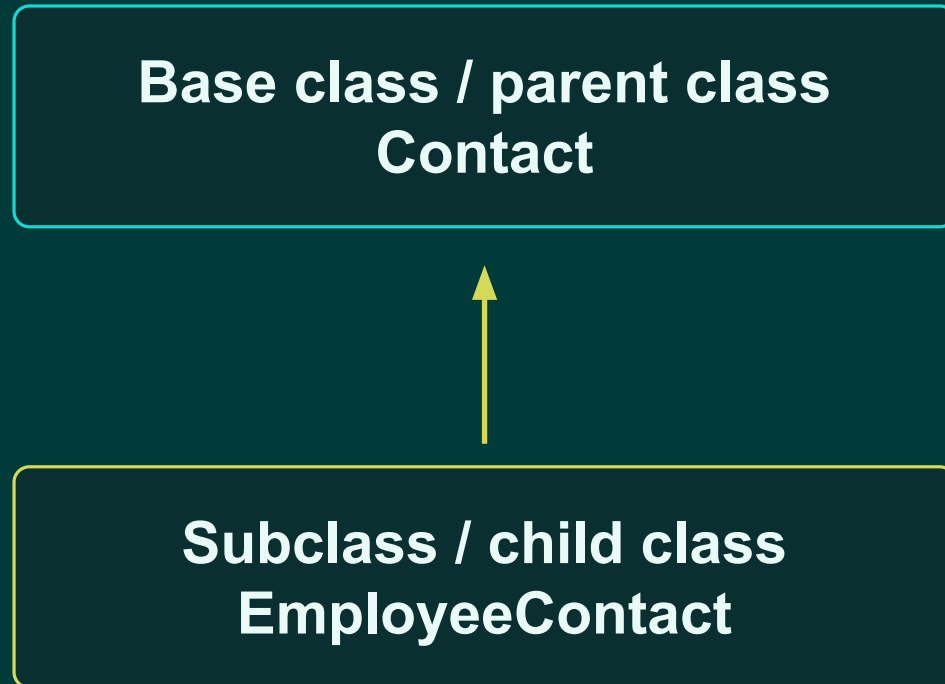
    def __init__(self, name, email):
        self.name = name
        self.email = email

class EmployeeContact(Contact):
    pass
```



Base Class and Subclass

Inheritance vocabulary appears often in documentation.



An EmployeeContact receives shared behavior from Contact.

Then it can add employee-specific fields or methods.

Inheritance is often described as an “is-a” relationship.

Inherited Initialization

A subclass can use behavior defined by the parent class.

```
class EmployeeContact(Contact):  
    pass  
  
employee = EmployeeContact(  
    "Alice",  
    "alice@example.com"  
)  
  
print(employee.name)
```

EmployeeContact did not define
__init__ yet.



Python can use Contact.__init__
for the shared fields.

Adding Specialized Information

Use parent initialization, then add subclass-specific data.

```
class EmployeeContact(Contact):  
  
    def __init__(  
        self,  
        name,  
        email,  
        employee_id  
    ):  
        super().__init__(  
            name,  
            email  
        )  
  
        self.employee_id = employee_id
```

EmployeeContact initialization



Use Contact initialization



Add employee_id

super() Mental Model

For now, read super() as “work with the parent-class part.”

```
super().__init__(  
    name,  
    email  
)
```

Parent class portion



Shared setup logic

Do not memorize every detail today. Recognize that super() helps a subclass reuse parent behavior.

Method Overriding

A subclass can replace a method with its own implementation.

base class

```
class Contact:

    def display(self):
        print(self.name)
        print(self.email)
```

subclass

```
class EmployeeContact(Contact):

    def display(self):
        print(self.name)
        print(self.email)
        print(self.employee_id)
```

EmployeeContact overrides display() to include employee_id.

Same Method, Different Object

Different objects can respond to the same operation in their own way.

```
contacts = [  
    Contact(...),  
    EmployeeContact(...),  
]  
  
for contact in contacts:  
    contact.display()
```

Contact.display()

EmployeeContact.display()

polymorphism
“same operation, different object
behavior”

Real Library Example: pathlib

Path objects already use class, attribute/property, and method ideas.

```
from pathlib import Path

path = Path("records.csv")

print(path.name)
print(path.suffix)
print(path.exists())
```

Path is a class

path is an object

name / suffix are attributes or properties

exists() is a method

Real Library Example: DataFrames

Object-oriented APIs are common because they group data and operations.

```
df = pandas.read_csv("records.csv")

df.head()
df.info()
df.sort_values("status")
df.dropna()

print(df.shape)
print(df.columns)
```

df is a DataFrame object

Methods operate on the table data

data + operations = convenient
interface

Classes Versus Functions

Both styles can be excellent when they make the code clearer.

procedural style

```
normalized =  
normalize_contact(contact)  
  
valid = validate_contact(normalized)  
  
summary = create_summary(normalized)
```

object-oriented style

```
contact.normalize()  
  
valid = contact.is_valid()  
  
summary = contact.create_summary()
```

The question is not “Which is always better?” The question is “Which is clearer here?”

When a Class Often Makes Sense

Classes can help when a concept has state and related behavior.

Related data

Related operations

Multiple instances

**State changes over
time**

Examples

Contact
BankAccount
Sensor
Order
DataPipeline

A class gives the concept a home: its data and its behaviors travel together.

When a Class May Be Unnecessary

Do not create a class merely because Python supports classes.

clear as a function

```
def celsius_to_fahrenheit(celsius):  
    return celsius * 9 / 5 + 32  
  
def normalize_status(status):  
    return status.strip().lower()
```

possibly unnecessary ceremony

```
class TemperatureConverter:  
    ...  
  
class StatusNormalizer:  
    ...
```

Simple functions are still good Python.

Guided Exercise Block

Build an address book that uses composition and inheritance.

30 minutes

1. Add Address

2. Add EmployeeContact

3. Override display()

4. Mixed collection

5. Choose a design

Goal: recognize has-a, is-a, super(), override, and same-method/different-object behavior.

Exercise 1 — Add an Address Class

Create an object that can be stored inside Contact.

7 minutes

```
class Address:

    def __init__(
        self,
        street,
        city,
        state,
        zip_code
    ):
        self.street = street
        self.city = city
        self.state = state
        self.zip_code = zip_code
```

Task
Modify Contact so it receives address.

```
print(alice.address.city)

# expected concept:
# Contact has an Address
```

Exercise 2 — Add EmployeeContact

Create a subclass that specializes Contact.

7 minutes

```
class EmployeeContact(Contact):  
  
    def __init__(  
        self, name, email,  
        address, employee_id  
    ):  
        super().__init__(  
            name, email, address  
        )  
  
        self.employee_id = employee_id
```

**EmployeeContact IS A
Contact**

It reuses shared contact setup,
then adds employee_id.

Exercise 3 — Override display()

The employee display should include employee_id.

6 minutes

```
# Contact
def display(self):
    print(self.name)
    print(self.email)

# EmployeeContact
def display(self):
    print(self.name)
    print(self.email)
    print(self.employee_id)
```

Task

Call display() on Alice and Bob.

Question

Which display method runs?

Exercise 4 — Mixed Object Collection

Use the same loop with different object types.

5 minutes

```
contacts = [  
    alice,  
    employee,  
]  
  
for contact in contacts:  
    contact.display()
```

Same line of code



Different object behavior

That is the basic idea of polymorphism.

Exercise 5 — Decide Whether a Class Helps

Choose function, dictionary, or class.

5 minutes

Miles → kilometers
Likely: function

500 contacts with
behavior
Reasonable: class

JSON configuration
Likely: dictionary

Bank accounts
Reasonable: class

Normalize one status value
Likely: function

There is usually more than one reasonable design. Explain your choice.

Applied Challenge — Address Book

Build a small object-oriented address book.

15 minutes

Required classes

Address

Contact

EmployeeContact

HAS-A
Contact has an
Address

IS-A
EmployeeContact is
a Contact

Tasks: create several objects, loop through all contacts, display each contact, mark favorites, print favorites, access address city, add employee_id, and override employee display.

Exit Check

Object-oriented Python is mostly ordinary Python with a few new relationship patterns.

**Class
blueprint**

**Object
instance**

**Attribute
stored data**

**Method
behavior**

**Composition
HAS-A
Contact has an Address**

**Inheritance
IS-A
EmployeeContact is a
Contact**

Read object-oriented APIs as: `object.attribute` and `object.method()` — data plus behavior from a class.